A Comprehensive Theoretical and Experimental Comparison of Dijkstra's Algorithm and Duan et al.'s SSSP Algorithms

Anmol, Tanmay Arora, Adhyyan, Pahul Veer Singh Gogna
Department of Computer Science & Engineering
Chitkara University
Punjab, India
anmol0115.be23@chitkara.edu.in, tanmay1305.be23@chitkara.edu.in,
adhyya0180.be23@chitkara.edu.in, pahul0508.be23@chitkara.edu.in

***Abstract*—Dijkstra's algorithm has been the practical method for single-source shortest paths for six decades. In 2025 Duan, Mao, Mao, Shu and Yin gave the first deterministic algorithm for directed graphs with real non-negative weights that is asymptotically faster on sparse graphs, and in 2026 Duan, Mao, Shu and Yin improved the bound with a second algorithm. Both results are theoretical. We compare the three algorithms under identical conditions: we contrast the two new algorithms component by component, implement both from the published pseudocode, validate them on 200 000 and 30 000 random graphs, and measure running time and machine-independent counts of comparisons, additions and edge scans on eleven real graphs and four synthetic families with up to four million vertices. Dijkstra's algorithm with a binary heap is the fastest on every instance, by a factor of 3.5 to 8.0 on the five primary road networks. On random and preferential-attachment graphs, whose search frontier is wide, the earlier of the two new algorithms needs fewer comparisons than Dijkstra's algorithm on large graphs, so its asymptotic advantage shows in the cost model of the theorems, yet it remains 1.5 times slower at four million vertices. The later algorithm removes most of the pivot-finding work of the earlier one but is the faster of the two on only one instance. Instrumented runs also show that a stale-key defect known for the earlier algorithm occurs in the published pseudocode of the later one, and that its base case can return a boundary that violates its own invariant; neither affected any computed distance.**

***Keywords*—single-source shortest paths, Dijkstra's algorithm, bounded multi-source shortest path, sorting barrier, experimental algorithmics, comparison-addition model**

I. Introduction

The single-source shortest path (SSSP) problem asks for the length of a shortest path from a source vertex s to every vertex of a directed graph $G = (V, E)$ with $n = |V|$ vertices, $m = |E|$ arcs and non-negative real arc weights. Dijkstra's algorithm [1] solves it in $O(m + n \log n)$ time when the frontier is kept in a Fibonacci heap [2] or a relaxed heap [3], in the comparison-addition model in which weights may only be added and compared. Because the algorithm settles vertices in order of distance, it sorts them as a by-product, and $n \log n$ was long regarded as a barrier for sparse graphs. Haeupler et al. [4] made the intuition precise: with a suitable heap Dijkstra's algorithm is universally optimal among algorithms that must output the vertices in distance order. The qualification matters, because an algorithm that returns only the distances is not bound by it.

Duan, Mao, Mao, Shu and Yin [5] exploited exactly this gap. Their deterministic algorithm runs in $O(m \log^{2/3} n)$ time and is the first to beat $O(m + n \log n)$ on sparse directed graphs with real weights. Its core is a recursive procedure, bounded multi-source shortest path (BMSSP), that never keeps the whole frontier sorted: a subroutine reduces the frontier to a small set of pivots, and a block-based structure hands out the smallest part of the frontier without ordering it internally. In 2026 Duan, Mao, Shu and Yin [6] improved the bound to $O(m\sqrt{\log n} + \sqrt{mn \log n \log\log n})$, which is $O(m\sqrt{\log n \log\log n})$ for sparse graphs, by replacing the Bellman–Ford style pivot search with local Dijkstra searches and a forest partition, and by simplifying the data structure. Neither paper gives its algorithm a name. Since both are built around the BMSSP procedure, we refer to the algorithm of Duan, Mao, Mao, Shu and Yin [5] as BMSSP-25 and to the algorithm of Duan, Mao, Shu and Yin [6] as BMSSP-26; the suffix is the year of publication.

Neither paper reports an implementation. Three facts make an experimental study worthwhile. First, the asymptotic gain is small: $\log^{1/3} n$ is 2.7 for $n = 2^{20}$ and only 4.0 for $n = 2^{64}$. Second, the constants are potentially large, since both algorithms scan arcs several times and maintain bespoke structures. Third, the preconditions are not free: both analyses assume a degree-bounded graph obtained by a transformation, and both assume that all path lengths are distinct, which must be enforced by tie-breaking. BMSSP-25 has been evaluated before [7]–[9]; to our knowledge no measurements of BMSSP-26 have been published in a peer-reviewed venue, and no earlier study reports counts in the model in which the bounds are stated.

This paper makes the following contributions.

- A component-by-component comparison of Dijkstra's algorithm, BMSSP-25 and BMSSP-26, including the values their parameters take at feasible sizes (Section III).

- Independent C++ implementations of both BMSSP algorithms, written from the pseudocode, together with randomized differential validation (Sections IV and VI-A).
- Two observations on the published pseudocode of BMSSP-26 made while validating: completed vertices can remain stored in the caller's structure, as in BMSSP-25, and the base case can return a boundary larger than a value it leaves behind (Section VI-A).
- Measurements of running time and of comparisons, additions and edge scans on five real road networks, six further real graphs of the SNAP collection and four synthetic families, with sweeps over the parameters k and t, the degree reduction, density and weight range (Section VI).

II. Related Work

A. Classical results

Fredman and Tarjan [2] obtained the $O(m + n \log n)$ bound for Dijkstra's algorithm with Fibonacci heaps, and Driscoll et al. [3] matched it with relaxed heaps. Experimental work on shortest-path algorithms has a long tradition [10], and in practice Dijkstra's algorithm is usually run with a binary heap, as in [7]. Faster algorithms were known only outside the setting considered here: Thorup's linear-time algorithm [11] needs undirected graphs and integer weights in the word RAM, and the algorithm of Pettie and Ramachandran [12] is for undirected graphs. For undirected real-weighted graphs Duan et al. [13] gave a randomized $O(m\sqrt{\log n \log\log n})$ algorithm, later derandomized by Yan [14]; a further improvement for undirected graphs appeared while this paper was being prepared [15].

B. Experimental studies of BMSSP-25

Castro, Clementino and de Freitas [7] implemented BMSSP-25 in C++ in three variants (worst case, constant degree, expected time) and compared the first with binary-heap Dijkstra on sparse random graphs with up to 2^{25} vertices, grids and U.S. road networks. They report that Dijkstra is three to four times faster on large instances and up to 27.5 times faster on small grids, estimate by curve fitting that a crossover would lie far beyond any feasible size, and find the constant-degree transformation counterproductive. They also identified a defect in the original analysis: a vertex that is completed inside a recursive call may stay in the data structure with an outdated key, so the sets returned by successive calls need not be disjoint; deleting the vertex on completion repairs it, and the original authors confirmed both problem and repair [7]. Their code is public [16]. Valko, Paranjpe and Marx Gómez [8] evaluated a Rust implementation on Lightning Network snapshots and likewise found Dijkstra faster. Makowski et al. [9] provide a Python package with measurements on grid and geospatial networks.

Several public repositories also implement BMSSP-25, and some report that it outperforms Dijkstra's algorithm [17], [18]. On inspection of their sources these reports concern variants that depart from the published algorithm. The optimized solver of [17], for example, replaces the recursion over levels, the pivot search and the block structure by a bucket queue of fixed bucket width and is timed for a single source–target pair on graphs with unit weights, where bucketing by distance is a linear-time method outside the comparison-addition model; the same repository reports its solver that follows the paper to be two to four times slower than Dijkstra. The benchmark of [18] likewise times source–target queries on unit-weight graphs against a library routine. We therefore do not regard these reports as measurements of the algorithm of [5], but we include graphs they use in our experiments (Section V).

The studies above report wall-clock time (and, in [7], memory) for BMSSP-25. They leave open how BMSSP-26 behaves, how much of the observed gap is additional algorithmic work rather than cost per operation, and how the result depends on graph structure beyond random graphs, grids and road networks.

III. The Three Algorithms

A. A common view

All three algorithms maintain labels $\widehat{d}[v] \geq d(v)$, where $d(v)$ is the true distance, and only lower a label by relaxing an arc (u, v), that is, by setting $\widehat{d}[v] \leftarrow \widehat{d}[u] + w_{uv}$. A vertex is *complete* when $\widehat{d}[v] = d(v)$. Following [6], each algorithm keeps a *frontier*: a set of vertices such that the shortest path to every incomplete vertex passes through a complete frontier vertex. An algorithm repeatedly identifies a set Z of complete vertices, relaxes the arcs leaving Z, and replaces Z in the frontier by the out-neighbours it has just improved. The algorithms differ in how they find Z.

Dijkstra's algorithm takes Z to be the single frontier vertex with the smallest label, which is necessarily complete. Finding it requires a priority queue over the whole frontier, and a frontier of $\Theta(n)$ vertices then costs $\Theta(\log n)$ comparisons per extraction.

Both BMSSP algorithms instead solve the subproblem BMSSP(l, B, S): given a level l, a bound B and a frontier S, complete all vertices v with $d(v) < B$ whose shortest path visits S. A call pulls the part S_i of its frontier that lies below a separator B_i, recurses on $(l-1, B_i, S_i)$, relaxes the arcs out of the returned complete set U_i, and repeats. A call may stop early (a *partial* execution) once it has completed enough vertices, returning a boundary $B' < B$ up to which its work is final. The whole problem is the call with $S = \{s\}$, $B = \infty$ at level $\lceil (\log n)/t \rceil$. Table I lists the components.

B. BMSSP-25: the algorithm of Duan, Mao, Mao, Shu and Yin

FindPivots of BMSSP-25 relaxes outward from S for k rounds, restricted to labels below B, and collects the touched vertices in W. If $|W|$ exceeds $k|S|$ the frontier is already small relative to the work ahead and every vertex of S becomes a pivot. Otherwise the tight arcs inside W form a forest, and only roots in S of trees with at least k vertices become pivots: a vertex whose shortest path leaves S through a smaller tree was completed by the k rounds. The pivots enter a structure D that returns, per Pull, at most $M = 2^{(l-1)t}$ keys with the smallest

TABLE I
COMPONENTS OF THE THREE ALGORITHMS

	Dijkstra [1], [2]	BMSSP-25 [5]	BMSSP-26 [6]
Running time	$O(m + n \log n)$	$O(m \log^{2/3} n)$	$O(m\sqrt{\log n} + \sqrt{mn \log n \log\log n})$
Output	distances in sorted order	distances	distances
Complete set Z	minimum of the frontier	set U_i returned by a recursive call	set U_i returned by a recursive call
Frontier reduction	none	pivots: roots of shortest-path trees with at least k vertices; at most $\lvert U\rvert/k$ of them	one representative (the minimum) per subtree of $\Theta(k)$ vertices of a forest spanning S
Pivot finding	–	k rounds of Bellman–Ford relaxation from S: $O(k)$ per vertex	local Dijkstra searches that stop after k vertices, then a linear-time partition: $O(\log k)$ per vertex
Partial sorting	heap (total order)	two block sequences; INSERT in $O(\log(N/M))$, BATCHPREPEND, PULL	one search tree of blocks; INSERT in $O(\log(N/M))$, linear-time MERGE and PULL
Hand-back after a partial call	–	caller re-inserts what it can reconstruct (BATCHPREPEND)	callee returns its remaining frontier D; caller merges it
Call stops when	frontier empty	$\lvert U\rvert \geq k2^{lt}$ or D empty	$\lvert U\rvert > t^3 2^{lt}$ or D empty
Base case ($l = 0$)	–	Dijkstra from one vertex, $k + 1$ vertices	Dijkstra from a set, $t^3 + 1$ vertices
Parameters	–	$k = \lfloor \log^{1/3} n \rfloor$, $t = \lfloor \log^{2/3} n \rfloor$	$t = \lceil \sqrt{\log n \log\log n/\delta} \rceil$, $k = \lceil t/\log t \rceil$
Degree assumption	none	in- and out-degree at most 2	degree at most δ, $3 \leq \delta \leq m/n$

values and a separator. Vertices improved by relaxing out of U_i are inserted individually if their new value is at least B_i and prepended in a batch if it lies in $[B'_i, B_i)$. The analysis [5] charges $O(k)$ per vertex and level for FINDPIVOTS and $O(t)$ per insertion, and balances the two with $k = \lfloor \log^{1/3} n \rfloor$ and $t = \lfloor \log^{2/3} n \rfloor$.

C. BMSSP-26: the algorithm of Duan, Mao, Shu and Yin

The observation behind BMSSP-26 is that the k relaxation rounds determine, for every vertex, the pivot tree to which it belongs, which is more than the recursion needs [6]. FINDPIVOTS now runs a Dijkstra search from each $x \in S$ that stops once it has discovered k vertices or touches a tree built by an earlier search, in which case the two are joined. A search that exhausts its reachable vertices before finding k puts x into a set Q and its vertices into W; they stay out of the recursion and are dealt with once at the end of the call. The remaining trees are cut into edge-disjoint subtrees with between k and $3k$ vertices [19], the frontier vertices of each subtree form a group P_j, and only the minimum p_j of each group is inserted into D. When p_j is pulled, the members of P_j below the separator join S_i by a scan of the group. The structure is a single search tree of blocks of $M = t\,2^{(l-1)t}$ keys, and a call returns its remaining frontier as such a structure, which the caller merges in linear time. The resulting time is $O(m(t+\log n \log t/(\delta t)))$ on a graph of maximum degree δ, minimized by $t = \lceil \sqrt{\log n \log\log n/\delta} \rceil$.

D. Preconditions

Distinct path lengths. Both analyses assume that no two paths have the same length. Both papers achieve this by comparing labels as tuples ⟨length, number of arcs, vertex, predecessor⟩ in lexicographic order. Every label then carries four fields and every comparison may inspect several of them, whereas Dijkstra's algorithm compares single numbers.

Bounded degree. BMSSP-25 is analysed on a graph in which every vertex has in- and out-degree at most two, obtained by replacing each vertex by a cycle of zero-weight arcs with one cycle vertex per incident arc [5], [19]. The result has up to $2m$ vertices and $3m$ arcs. BMSSP-26 generalizes the construction to cycles of $\lceil \Delta_v/(\delta - 2) \rceil$ vertices for a vertex with Δ_v incident arcs and requires $3 \leq \delta \leq m/n$. The value proposed for δ is $\min\{m/n, \log\log n\}/4$ [6], which reaches the required minimum of three only when $\log\log n \geq 12$, that is, for $n \geq 2^{4096}$; the authors note that constants may be adjusted by implementers. For every feasible input the prescription therefore amounts to $\delta = 3$ (or to no reduction when $m/n < 3$), and we use $\delta = 3$ in the formula for t throughout.

TABLE II
PRESCRIBED PARAMETERS FOR CONCRETE n ($\delta = 3$). LEVELS = $\lceil \log_2 n/t \rceil$; GAIN = FACTOR BY WHICH THE BOUND OF THE ALGORITHM IMPROVES ON $n \log n$ WITH UNIT CONSTANTS

	BMSSP-25				BMSSP-26				
n	k	t	levels	gain	k	t	levels	t^3	gain
2^{10}	2	4	3	2.15	2	4	3	64	1.74
2^{20}	2	7	3	2.71	3	6	4	216	2.15
2^{30}	3	9	4	3.11	3	8	4	512	2.47
2^{40}	3	11	4	3.42	3	9	5	729	2.74
2^{64}	4	16	4	4.00	4	12	6	1 728	3.27

E. Parameters at feasible sizes

Table II evaluates the prescribed parameters. For all sizes that fit in memory today BMSSP-25 uses $k \in \{2, 3\}$, so its FINDPIVOTS performs two or three relaxation rounds, and three or four recursion levels; BMSSP-26 uses $k \in \{2, 3\}$ as well, so each local search stops after two or three vertices, and its base case is a Dijkstra search over a few hundred vertices. The last two columns give the factor by which each bound improves on $n \log n$ for sparse graphs when all hidden constants are set to one. Read this way the bound of BMSSP-25 is the smaller of the two until $(\log\log n)^3 < \log n$, that is, until $\log n$ exceeds 982. Constants decide the comparison at any feasible size, which is what the experiments measure.

IV. IMPLEMENTATION

All algorithms are implemented in C++17 in a single code base and share the graph representation (compressed sparse

rows), the label type and the timing harness. The BMSSP-25 and BMSSP-26 implementations were written from the pseudocode of [5] and [6]; no code from other implementations was used.

Dijkstra. Two variants are provided: a binary heap [20] of (key, vertex) pairs with lazy deletion, the usual practical implementation, which we call Dijkstra, and an indexed binary heap with DecreaseKey, which we call Dijkstra-DK. Both operate on plain path lengths. The first is the baseline in all comparisons; on the two instances on which we checked (SF and a random graph with 2^{20} vertices) it was slightly faster than `std::priority_queue`.

Labels. The BMSSP implementations store a 16-byte label (length, number of arcs, predecessor) per vertex and compare the tuples described in Section III. Arc weights are 64-bit integers, so sums are exact and the results of different algorithms can be compared for equality.

Partial-sorting structures. Blocks are arrays with constant-time deletion by swapping with the last element. The block index is a red-black tree (`std::map`) keyed by block upper bounds, as proposed in both papers [21]. A direct-address table per recursion level maps a vertex to its block and position, as in [7]; this is possible because calls on one level run one after another. Selection of the element of a given rank, needed by Pull, block splits and BatchPrepend, uses quickselect that switches to median-of-medians pivots [22] once a linear work budget is exceeded [23], so the worst case stays linear. The base case of BMSSP-26 uses a balanced search tree (`std::set`), as its pseudocode prescribes, and the local searches of both algorithms use binary heaps instead of Fibonacci heaps; with at most $k + 1$, respectively $k + \delta$, entries this does not change any bound that matters at the sizes considered.

Sets. The sets U returned by recursive calls are stored as contiguous ranges of one global array, so returning a set costs nothing and a caller's set is the concatenation of its children's ranges. Membership tests use time-stamped arrays.

Deviations from the pseudocode. Three places are implemented differently from the printed pseudocode. They are switchable, and Section VI-A reports what each of them changes. (i) In both algorithms a vertex is deleted from the caller's D when a recursive call reports it complete. For BMSSP-25 this is the repair of [7]. (ii) The base case of BMSSP-26 returns the smallest value remaining in its structure after the loop instead of the separator of its last Pull. (iii) Line 28 of the main procedure of BMSSP-26 selects a new representative only for groups P_j that are still non-empty, as the accompanying text of [6] says; a group can be emptied by later vertices of the same U_i after it was scheduled for re-selection.

Counters. A second build of the same sources increments counters for label or key comparisons, additions $\widehat{d}[u] + w_{uv}$, edge scans (split by where they occur), and structure operations. Comparisons include those made inside heaps, search trees and selection. The timing build contains no instrumentation.

TABLE III
Road networks (arcs counted in both directions). Front.: largest number of vertices held at once by Dijkstra's heap, in percent of n. Last columns: prescribed $k/t/$levels

	n	m	max deg.	m/n	front. (%)	BMSSP-25 $k/t/$levels	BMSSP-26 $k/t/$levels
OL	6 105	14 070	5	2.30	1.7	2/5/3	2/4/4
TG	18 263	47 748	8	2.61	1.0	2/5/3	3/5/3
CAL	21 048	43 386	8	2.06	0.3	2/5/3	3/5/3
SF	174 956	446 002	8	2.55	0.3	2/6/3	3/5/4
NA	175 813	358 358	14	2.04	0.1	2/6/3	3/5/4

The implementations, the experiment drivers, the raw measurements and the scripts that regenerate every table and figure of this paper from them are publicly available [24].

V. Experimental Setup

Road networks. The real inputs are the five road networks of the SSTD 2005 collection [25], obtained from the public mirror [26]: Oldenburg (OL), San Joaquin County (TG), California (CAL), San Francisco (SF) and North America (NA); OL, TG and SF originate from Brinkhoff's generator data [27]. Each file lists a road segment once with its Euclidean length. All five networks are connected when segments are read as undirected, but a search that follows segments only in their listed direction reaches as few as six vertices from vertex 0 of CAL, so every segment is entered as two opposite arcs. Lengths are given with six decimals and are multiplied by 10^6 to obtain exact integers. Table III lists the resulting graphs.

SNAP graphs. To cover larger real graphs of other kinds we add six graphs of the SNAP collection [28], all of which are also used in [17]: the web graphs web-Stanford and web-Google, the road networks of Pennsylvania, Texas and California, and the Pokec social network (Table IX). The road files list both directions of every road; all files are used as given. They carry no weights, so every graph is run twice, with unit weights as in [17] and with uniform random integer weights in $[1, 2^{20}]$. Since a random vertex of a directed web graph may reach almost nothing, sources are random vertices that reach at least a quarter of the graph.

Synthetic families. Four families with n from 2^{10} to 2^{22} extend the range of sizes and structures: (i) random digraphs with a random Hamiltonian cycle and $2n$ further uniformly random arcs ($m = 3n$); (ii) square grids with arcs in both directions; (iii) preferential-attachment graphs [29] with two links per new vertex, entered in both directions, which have a heavy-tailed degree distribution; and (iv) hub graphs, which are random digraphs in which the source additionally has an arc to every other vertex whose weight exceeds every ordinary weight, so that the search starts with all n vertices in its frontier. Weights are uniform integers in $[1, 2^{20}]$ unless varied. Two sweeps on random digraphs with $n = 2^{18}$ vary the average out-degree from 2 to 32 and the largest weight from 1 (all paths of equal hop count tie) to 2^{30}.

Measures. We report wall-clock time of one SSSP computation and, from the instrumented build, the number of

TABLE IV
RANDOMIZED VALIDATION WITH THE PRESCRIBED PARAMETERS: NUMBER OF RUNS IN WHICH AN EVENT OCCURRED AT LEAST ONCE

Graphs	Variant	Runs	Wrong dist.	Stale key	Dupl. compl.	Bound viol.
$n \le 16$	BMSSP-25, as printed	200 000	0	5 293	4 788	0
	BMSSP-25, with deletion	200 000	0	5 293	0	0
	BMSSP-26, as printed	200 000	0	3 475	3 475	8 939
	BMSSP-26, both repairs	200 000	0	3 475	0	0
$n \le 400$	BMSSP-25, as printed	30 000	0	10 402	9 343	0
	BMSSP-25, with deletion	30 000	0	10 402	0	0
	BMSSP-26, as printed	30 000	0	10 510	10 510	1 435
	BMSSP-26, both repairs	30 000	0	10 510	0	0

comparisons, additions and edge scans divided by m. A run includes initialization of the labels but not graph loading or allocation of working memory, which is done once per graph.

Protocol. On each road network every algorithm is run from the same 20 uniformly random sources, nine times each; SNAP instances use ten sources and three repetitions, synthetic instances three random sources (the hub for family (iv)) and five repetitions. Every run is a complete single-source computation over the whole graph. The degree-reduction and parameter experiments use ten and five sources per network. For each source we take the median over repetitions. Ratios to Dijkstra are formed per source and summarized by their geometric mean. The distances of every timed run are compared with those of Dijkstra's algorithm. Unless stated otherwise the BMSSP algorithms run on the input graph as given, without degree reduction, with the parameters of Table III and with deviations (i)–(iii).

Platform. Apple M1 (8 cores, 8 GB), macOS 26.1, Apple clang 17.0.0 with `-O3`; all runs are single-threaded.

VI. RESULTS

A. Validation and observations on the pseudocode

Every variant was run on 200 000 random graphs with at most 16 vertices and 30 000 random graphs with at most 400 vertices, drawn from six shapes (sparse, dense, paths with chords, stars, trees with extra arcs, grids) and five weight ranges including all-zero weights, and compared with a Bellman–Ford computation [30]. Each graph was solved with the prescribed parameters, with random small k and t that force partial executions, and after degree reduction with $\delta \in \{3, 4, 5\}$. No variant, repaired or as printed, returned a wrong distance in any of the 3 400 000 and 510 000 runs. Table IV summarizes what the instrumentation recorded with the prescribed parameters.

Stale keys. A key stored in D keeps the value it had when inserted, while the label of its vertex can decrease during a recursive call. For BMSSP-25 this is the defect reported in [7]: in our runs without deletion a vertex was reported complete a second time within one call in 4 788 of the small and 9 343 of the medium instances, the smallest having 9 vertices. The same happens in BMSSP-26. The proof of Lemma 9 in [6] (lemma numbers refer to the proceedings version) states that the returned set U_i is not in D and needs no removal; in 3 475 small and 10 510 medium instances at least one vertex of U_i was still stored in the caller's D when the sub-call returned. Without deletion such a vertex is pulled again later with its old key and its already completed subtree is explored again, so the sets U_i of one call are not disjoint (3 475 and 10 510 instances, smallest with 12 vertices), contrary to Lemma 10 of [6], on which the running-time analysis relies. Deleting the vertices of U_i from D after the merge removes every such event. We have not contacted the authors of [6]; the statement here is limited to what the instrumented runs show.

Boundary returned by the base case of BMSSP-26. Algorithm 4 of [6] returns the separator delivered by its last PULL, which is computed before the arcs of the pulled vertex are relaxed. If the loop then ends because t^3+1 vertices have been extracted, those relaxations may have inserted values below the returned B', contradicting the requirement $B' \le d_B[D]$ of Lemma 9, or the structure may be non-empty although $B' = B$ signals a full execution. A directed path with $t^3 + 2$ vertices searched from its first vertex is the simplest example: the last PULL empties the structure and returns B, and relaxing the pulled vertex then inserts its successor. The event occurred in 8 939 small and 1 435 medium instances (smallest: 10 vertices). Returning the smallest remaining value after the loop restores the invariant.

On the road networks these events are rare. Without deletion, repeated completions amount to at most 1.0% of n for BMSSP-25 and at most 0.9% for BMSSP-26, and the running times of printed and repaired variants agree within measurement noise. The repairs matter for the validity of the analysis rather than for speed on these inputs.

B. Road networks: running time

Table V gives the main comparison. Dijkstra's algorithm is faster than both BMSSP algorithms on all five networks from every source: by a factor between 3.5 and 8.0 against BMSSP-25 and between 4.7 and 6.7 against BMSSP-26. The two Dijkstra variants are within about ten percent of each other. BMSSP-26 is the faster of the two new algorithms on one network only (CAL) and practically ties on NA; these are the two networks with the smallest average degree (Table III), and on them BMSSP-25 needs the most comparisons (Table VI). Over the individual start vertices the ratio ranges from 3.2 to 9.5 for BMSSP-25 and from 4.3 to 7.3 for BMSSP-26.

The largest ratios are a property of two particular inputs and not of road networks in general. In CAL and NA 93.5% and 94.8% of the vertices have exactly two neighbours, so these networks are long chains of shape points joined at few junctions, against 17% to 53% in the other three. On the two chain-like networks BMSSP-25 is 6.7 to 8.0 times slower than Dijkstra, on the other three 3.5 to 5.1 times, and on the three much larger SNAP road networks (Section VI-H) 2.6 to 3.0 times. The latter values are close to the factor of three to four reported by [7] for large road networks. Size matters as well: on instances with a few thousand vertices all ratios are higher, in our measurements (Fig. 3) as in theirs.

TABLE V

ROAD NETWORKS: MEDIAN TIME OF ONE SSSP COMPUTATION IN MILLISECONDS, AND RATIO TO DIJKSTRA (GEOMETRIC MEAN OF THE PER-SOURCE RATIOS). DIJKSTRA: BINARY HEAP WITH LAZY DELETION; DIJKSTRA-DK: INDEXED HEAP WITH DECREASEKEY

	Dijkstra	Dijkstra-DK		BMSSP-25		BMSSP-26	
	ms	ms	ratio	ms	ratio	ms	ratio
OL	0.220	0.223	1.02	1.083	5.08	1.285	5.86
TG	0.847	0.846	1.00	3.492	4.08	3.969	4.68
CAL	0.592	0.620	1.05	4.775	7.95	3.153	5.31
SF	8.495	9.157	1.07	30.002	3.50	40.222	4.68
NA	5.609	6.295	1.11	36.963	6.68	39.088	6.73

TABLE VI

ROAD NETWORKS: COMPARISONS (CMP.), ADDITIONS (ADD.) AND EDGE SCANS (SCANS) PER ARC, MEAN OVER SOURCES. DIJKSTRA'S ALGORITHM PERFORMS EXACTLY ONE ADDITION AND ONE EDGE SCAN PER ARC

	Dijkstra	BMSSP-25			BMSSP-26		
	cmp.	cmp.	add.	scans	cmp.	add.	scans
OL	6.66	13.05	3.94	4.71	14.30	5.13	5.13
TG	6.97	11.58	3.67	4.56	12.07	4.03	4.03
CAL	6.12	16.29	4.43	4.77	12.76	4.02	4.02
SF	7.99	11.12	3.32	4.25	13.06	5.08	5.08
NA	7.41	15.53	4.13	4.68	15.64	5.06	5.06

C. Road networks: operation counts

Table VI separates algorithmic work from cost per operation. Dijkstra's algorithm scans every arc once and performs one addition per arc. BMSSP-25 scans each arc between 4.3 and 4.8 times and BMSSP-26 between 4.0 and 5.1 times, and since each scan evaluates $\widehat{d}[u] + w_{uv}$, additions grow by 3.3 to 4.4 times and 4.0 to 5.1 times respectively. More notably, on these networks the new algorithms also perform *more comparisons* than Dijkstra's algorithm: 1.39 to 2.66 times as many for BMSSP-25 and 1.63 to 2.15 times as many for BMSSP-26. Dijkstra's heap never holds more than 1.7% of the vertices of a road network (Table III), so there is little ordering work to save, and what is saved is more than offset by the comparisons that accompany the repeated relaxations. Section VI-G shows that this changes on graphs with a wide frontier.

Fig. 1 shows where the scans occur and what BMSSP-26 changes. In BMSSP-25, FINDPIVOTS accounts for more than one scan per arc on every network: its relaxation rounds touch each arc about as often as Dijkstra's whole computation. In BMSSP-26 the local searches cost a small fraction of that, so the refinement achieves what it was designed for. The gain is offset in the relaxation loops: every level of the recursion relaxes all arcs leaving the sets it receives, and with the prescribed parameters BMSSP-26 has a level more than BMSSP-25 on OL, SF and NA (Table III), the topmost of which merely passes the single source down and scans every arc once more on the way back.

Dividing the time ratio by the ratio of comparisons plus additions gives the relative cost per counted operation: 2.1 to 2.9 for BMSSP-25 and 2.3 to 2.7 for BMSSP-26. The slowdown thus splits into two factors of similar size: 1.6 to 2.9 (BMSSP-25) and 2.0 to 2.5 (BMSSP-26) times more work

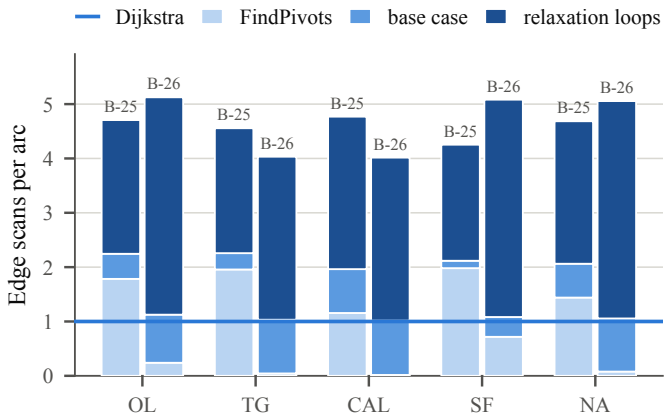


Fig. 1. Edge scans per arc on the road networks for BMSSP-25 (bars marked B-25) and BMSSP-26 (B-26), split by where they occur. Dijkstra's algorithm scans every arc exactly once.

TABLE VII

DEGREE REDUCTION: GROWTH OF THE GRAPH FOR $\delta = 3$ AND TIME RELATIVE TO DIJKSTRA ON THE ORIGINAL GRAPH

	growth		Dijkstra	BMSSP-25		BMSSP-26		
	n	m	δ=3	orig.	δ=3	orig.	δ=3	δ=4
OL	4.61	3.00	3.8	5.0	21.9	5.9	11.9	9.8
TG	5.23	3.00	3.9	4.1	21.1	4.6	14.0	9.9
CAL	4.12	3.00	3.6	7.8	19.5	5.3	13.7	8.7
SF	5.10	3.00	4.6	3.6	20.4	4.7	15.0	10.9
NA	4.08	3.00	4.5	6.6	24.9	6.7	19.5	11.7

in the comparison-addition model, and a higher cost of each operation that the model does not see: four-field labels instead of single numbers, bookkeeping of blocks and tables, and less regular memory access.

D. Degree reduction

Table VII reports the effect of running on the degree-reduced graph that the analyses assume. With $\delta = 3$ the graph has $2m$ vertices and $3m$ arcs, between 4.1 and 5.2 times the original number of vertices. Both BMSSP algorithms become 2.5 to 5.6 times (BMSSP-25) and 2.0 to 3.2 times (BMSSP-26) slower than on the original graph, and Dijkstra's algorithm itself is several times slower on the transformed input. The road networks already have maximum degree at most 14, so the transformation buys nothing here, in line with the finding of [7] for BMSSP-25. A larger bound $\delta = 4$ reduces the penalty for BMSSP-26 but does not remove it.

E. Sensitivity to the parameters k and t

The parameters are prescribed for asymptotic reasons, and both papers leave their constants open. Fig. 2 shows the running time on SF for all combinations of $k \in \{1, \ldots, 6\}$ and $t \in \{2, \ldots, 10\}$, with the number of levels following from t. Three observations hold on both SF and NA. First, $k = 1$ is uniformly poor: without frontier reduction every frontier vertex goes through the recursion, and on SF the algorithms are up to 14.1 (BMSSP-25) and 12.8 (BMSSP-26) times slower than Dijkstra. Second, the prescribed values are good but not

the best. For BMSSP-25 a larger k helps, which is consistent with the structure of these networks: with an average degree between 2.0 and 2.6 they consist largely of chains, and more relaxation rounds complete longer chains inside FindPivots. The fastest setting on SF is $k = 5$, $t = 6$ (12% faster than prescribed) and on NA $k = 6$, $t = 5$ (39% faster). For BMSSP-26 the fastest settings use $t = 9$ on both networks, which gives two levels instead of four, and are 11% (SF) and 31% (NA) faster. Third, no setting changes the conclusion: the best ratios to Dijkstra over the whole grid are 3.05 and 4.09 on SF and 3.94 and 4.72 on NA.

The topmost level of BMSSP-26 deserves a remark. The call one level below the top may complete up to $t^3 2^{(L-1)t}$ vertices, where $L = \lceil \log n/t \rceil$, and this is at least n whenever $t^3 \geq 2^t$, that is, for $t \leq 9$. For every n up to 2^{40} (Table II) the top call therefore pulls the source, receives all of V from its single sub-call and relaxes every arc once more without effect. This is visible in the counts: BMSSP-26 scans every arc once in a base case and once per level.

F. Scaling on synthetic families

Fig. 3 shows the time relative to Dijkstra as n grows from 2^{10} to 2^{22} (random and grid) or 2^{21} (preferential attachment and hub). On random digraphs the ratio falls steadily, from 5.9 at $n = 2^{10}$ to 1.5 at $n = 2^{22}$ for BMSSP-25 and from 6.8 to 1.8 for BMSSP-26. Preferential-attachment graphs behave alike (1.2 and 2.0 at $n = 2^{21}$); the heavy-tailed degree distribution makes no visible difference. On grids the improvement stops early: from $n = 2^{14}$ on both algorithms stay between 2.7 and 3.8 times slower than Dijkstra. On hub graphs BMSSP-25 improves to 1.7 while BMSSP-26 is still 3.9 times slower at the largest size. BMSSP-26 was the slower of the two new algorithms on all 44 synthetic instances, and neither overtakes Dijkstra anywhere in the tested range. The curves are not smooth because the prescribed k, t and number of levels are rounded and change in steps. Above 2^{20} vertices the two Dijkstra variants themselves separate, the DecreaseKey variant becoming up to 2.1 times slower, which indicates that at these sizes all algorithms are governed by memory-hierarchy effects that the cost model does not capture.

Fig. 4 shows the same instances in the cost model of the theorems, and here the picture differs between families. On random and preferential-attachment graphs Dijkstra's comparisons per arc grow linearly in $\log n$, by 0.80 and 0.67 per doubling of n with the lazy heap and by 0.67 and 0.50 with DecreaseKey. The counts of the BMSSP algorithms are nearly flat, between 10 and 12 comparisons per arc for BMSSP-25 and between 12 and 15 for BMSSP-26 on random graphs. The curves therefore cross: BMSSP-25 needs fewer comparisons than binary-heap Dijkstra at every tested size from $n = 2^{14}$ on for random graphs and from 2^{16} on for preferential-attachment graphs (2^{17} and 2^{20} against the DecreaseKey variant), and fewer comparisons plus additions from 2^{16} and 2^{18} on. BMSSP-26 falls below Dijkstra's comparison count at $n = 2^{20}$ in both families. This is the asymptotic advantage of the new algorithms appearing at feasible sizes, in the model in which it is proved. It does not carry over to running time: at $n = 2^{20}$ on random graphs BMSSP-25 executes 11.9 comparisons and additions per arc against 16.5 for Dijkstra and still takes 1.7 times as long. On grids and on hub graphs there is no crossing, for two different reasons discussed next.

G. Graph structure: frontier width and maximum degree

Two properties of the input explain the differences between the families.

Frontier width. We measure the width of Dijkstra's frontier by the largest number of vertices its indexed heap holds at any time. On the road networks this is between 0.1% and 1.7% of n (Table III), and on grids it shrinks from 7.5% at $n = 2^{10}$ to 0.1% at $n = 2^{22}$. The heap is small, extractions are cheap, and there is little ordering work for a cleverer algorithm to save; Dijkstra's algorithm makes fewer comparisons than both new algorithms at every size. On random and preferential-attachment graphs the frontier holds about 33% and 51% of the vertices at every size, Dijkstra's algorithm pays the logarithmic factor in full, and the counts of the new algorithms fall below it. Running time orders these inputs the same way: the smallest ratios to Dijkstra occur on random and preferential-attachment graphs, larger ones on grids and road networks.

Maximum degree. On hub graphs the frontier is as wide as it can be, since all n vertices are labelled as soon as the source is settled. Nevertheless both BMSSP algorithms need more comparisons than Dijkstra's algorithm, by a factor of 2.4 to 2.9 for BMSSP-25, increasing with n, and of 1.9 to 2.2 for BMSSP-26 (Fig. 4, right). The counters show why. The $n - 1$ arcs of the hub are relaxed together after the first base case. In BMSSP-25 they all fall into the range that is batch-prepended at level one, where a block holds a single key, so the structure sorts them by repeated median selection; in BMSSP-26 they are inserted into the search tree of the base case and handed upwards by merges. Either way the algorithm orders the entire frontier, at a higher cost per key than a heap. The analyses exclude this situation by the degree bound, which limits the number of keys one completed vertex can generate. Table VIII confirms that the bound is what is missing. After degree reduction with $\delta = 3$ the instance has eight times as many vertices and satisfies the precondition, and on it BMSSP-26 needs fewer than half the comparisons of Dijkstra's algorithm (55 against 118 per arc of the original graph) and BMSSP-25 fewer as well (98). The advantage is real but expensive: Dijkstra's algorithm on the original graph needs only 14 comparisons per arc, the transformation alone makes it 6.7 times slower, and on the transformed graph BMSSP-25 and BMSSP-26 still take 2.9 and 2.5 times as long as Dijkstra's algorithm on the same graph.

H. Web, road and social graphs

Table IX and Fig. 5 repeat the comparison on six larger real graphs. Dijkstra's algorithm is the fastest on all twelve instances: BMSSP-25 is between 1.3 and 3.2 and BMSSP-26 between 1.9 and 4.5 times slower, and BMSSP-26 is the faster of the two on none of them. The closest result is Pokec with



Fig. 2. Running time relative to Dijkstra on SF for every combination of k and t (lighter is faster). Solid box: parameters prescribed by the respective paper; dotted box: fastest combination.

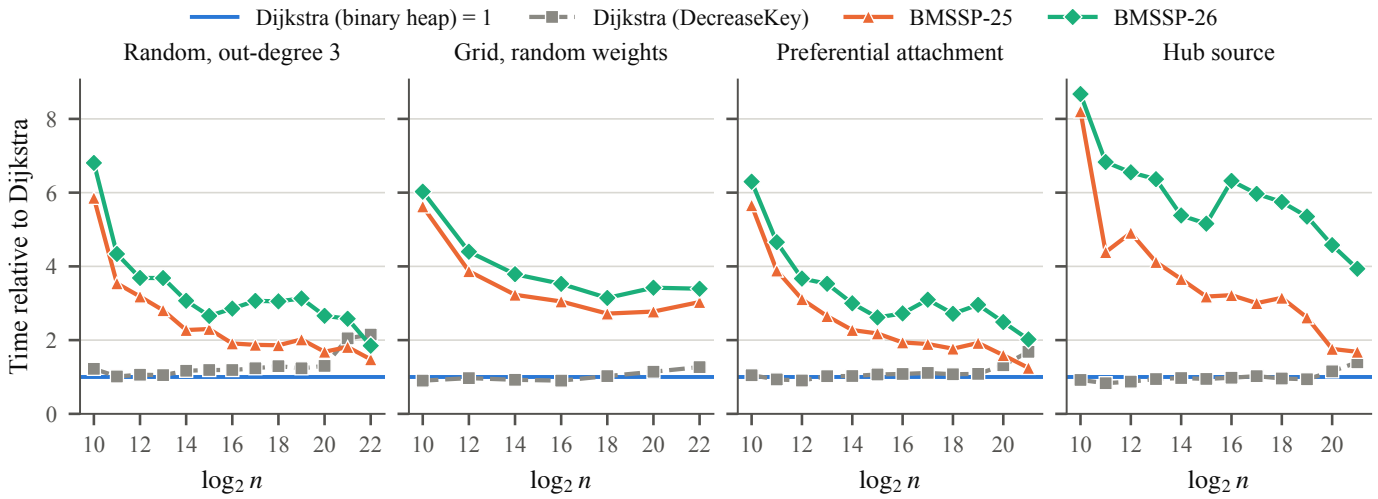


Fig. 3. Running time relative to Dijkstra with a binary heap on four synthetic families as a function of n (geometric mean over sources).

TABLE VIII
HUB GRAPH WITH $n = 2^{18}$ AS GIVEN AND AFTER DEGREE REDUCTION. CMP.: COMPARISONS PER ARC OF THE ORIGINAL GRAPH; TIME: RELATIVE TO DIJKSTRA ON THE GRAPH AS GIVEN

		Dijkstra		BMSSP-25		BMSSP-26	
Graph	vertices	cmp.	time	cmp.	time	cmp.	time
as given	262 144	14.0	1.0	37.5	2.9	26.0	6.4
$\delta = 3$	2 097 150	118.2	6.7	97.9	19.3	55.4	17.1

random weights, the densest graph having the widest frontier. In case of having unit weights, the setup in which [17] reports its simplified solver ahead of Dijkstra, the new algorithms are 2.2 to 3.2 (BMSSP-25) and 3.0 to 4.5 (BMSSP-26) times slower; we observed no practical advantage on any of the graphs that we chose.

The counts are similar to the pattern of Section VI-G. The frontiers of the three road networks are at most 0.3% of n, and both the published algorithms require more comparisons than Dijkstra's algorithm. The web and social graphs having frontiers between 13% and 50% of n; with random weights BMSSP-25 need fewer comparisons than the lazy-heap baseline on two of them (Google and Pokec), the two of the largest. Two qualifications apply. Measured against the DECREASEKEY variant, whose heap never exceeds n entries therefore is much cheaper on graphs which are dense, BMSSP-25 needs more comparisons on all of the twelve instances. And with unit weights, where Dijkstra's algorithm needs few comparisons per arc, the new algorithms need more of them on every graph (1.03 to 1.65 times for BMSSP-25).

I. *Density and weight range*

Two sweeps on random digraphs with $n = 2^{18}$ vary further properties. On increasing the average out-degree from 2 to 8, the ratio to Dijkstra from 2.2 to 1.5 for BMSSP-25 and from

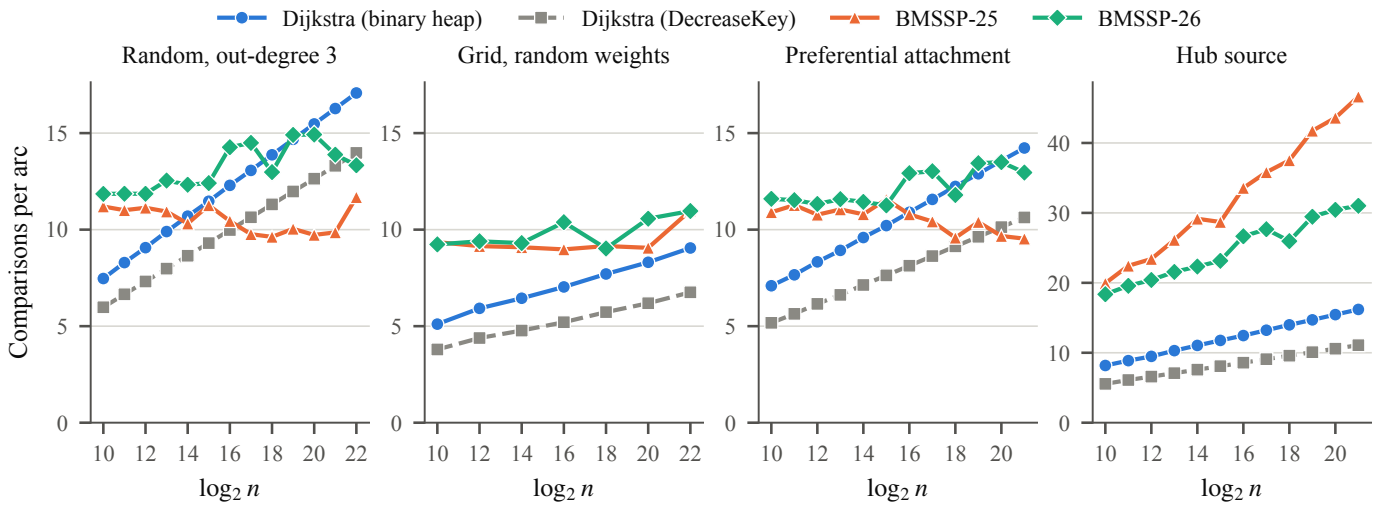


Fig. 4. Comparisons per arc on the synthetic families as a function of n. The hub family has its own vertical scale.

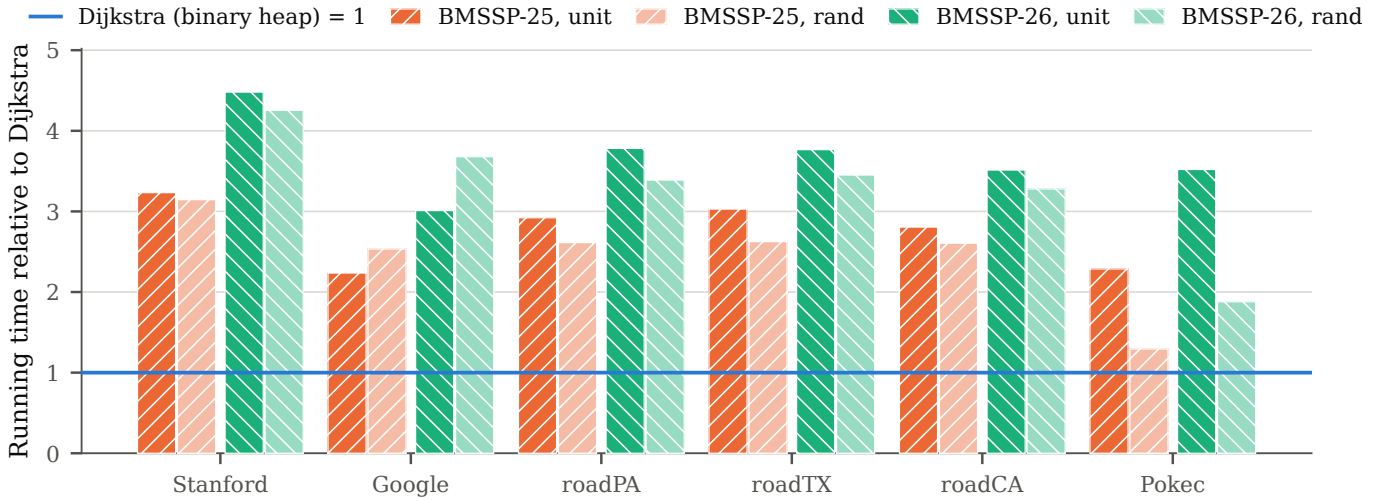


Fig. 5. Running time relative to Dijkstra with a binary heap on the six SNAP graphs, with unit and with random weights (geometric mean over ten sources). Dijkstra's algorithm is the fastest on all twelve instances.

TABLE IX

SNAP GRAPHS. FRONT.: PEAK FRONTIER OF DIJKSTRA IN PERCENT OF n (RANDOM WEIGHTS). FOR EACH OF THE TWO NEW ALGORITHMS: RUNNING TIME RELATIVE TO DIJKSTRA WITH UNIT WEIGHTS (UNIT) AND WITH RANDOM WEIGHTS (RAND.), AND COMPARISONS RELATIVE TO DIJKSTRA WITH RANDOM WEIGHTS (CMP.)

	n	m	front.	BMSSP-25			BMSSP-26		
	(10^3)	(10^6)	(%)	unit	rand.	cmp.	unit	rand.	cmp.
Stanford	282	2.31	13.1	3.2	3.1	1.20	4.5	4.3	1.52
Google	876	5.11	15.5	2.2	2.5	0.79	3.0	3.7	1.23
roadPA	1088	3.08	0.3	2.9	2.6	1.10	3.8	3.4	1.25
roadTX	1380	3.84	0.3	3.0	2.6	1.10	3.8	3.5	1.24
roadCA	1965	5.53	0.3	2.8	2.6	1.09	3.5	3.3	1.22
Pokec	1633	30.62	50.3	2.3	1.3	0.80	3.5	1.9	1.29

3.6 to 2.3 for BMSSP-26 lowers; beyond that the ratio remains roughly constant. Graphs which are denser spend a larger share of the time in plain edge scans, which is performed by all algorithms, but the BMSSP algorithms scan each arc three to four times, because of which the ratio does not approach one. The weight range has little influence: from unit weights, where all paths with the same number of arcs tie and the tie-breaking rule decides many comparisons, to weights up to 2^{30} the ratio stays between 1.9 and 2.3 for BMSSP-25 and between 2.8 and 3.8 for BMSSP-26.

J. Memory

The BMSSP implementations keeps a one direct-address table for each recursion level in addition to the labels. On the random graph with $n = 2^{22}$ the peak resident set of the whole process, which includes the graph and the reference solution, is 501 MB for Dijkstra, 859 MB for BMSSP-25 and 1170 MB for BMSSP-26, a factor of 1.7 and 2.3.

VII. Discussion

What BMSSP-26 gains over BMSSP-25? The redesigned FindPivots does what is promised. On the road networks it accounts for 0.02 to 0.72 edge scans per arc, against 1.16 to 1.98 for the relaxation rounds of BMSSP-25. The savings are lost by the side effects of the new parameter setting. The value of t is smaller than in BMSSP-25 (Table II), which gives more levels, and every level relaxes all arcs once: the relaxation loops cost 3.0 to 4.0 scans per arc against 2.1 to 2.8. In addition the base case is now a search-tree Dijkstra over up to $t^3 + 1$ vertices instead of a heap with a small number of entries. In total BMSSP-26 performs more additions than BMSSP-25 on four of the five road networks and more comparisons on four of them, and it was slower on four of the five road networks and on all synthetic instances. When t is chosen freely the fastest configurations of BMSSP-26 have two levels (Fig. 2), which suggests that the balance between t and the cost of FindPivots that is optimal asymptotically is not the practical at these sizes.

Where the time goes. For the narrow-frontier inputs the new algorithms lose twice: they execute 1.6 to 2.9 (BMSSP-25) and 2.0 to 2.5 (BMSSP-26) times as many comparisons and additions as Dijkstra's algorithm on the road networks, and each one of these counted operations costs them 2.1 to 2.9 and 2.3 to 2.7 times as much time. For wide-frontier inputs of bounded degree the first handicap disappears as n grows, but the second remains and keeps the new algorithms behind. It is tied to features of the design that the cost model does not penalize: every arc is scanned three to five times instead of just once, labels are four-field tuples instead of just single numbers, and the block structures add indirection and bookkeeping. Reducing the number of passes over the arcs, for instance by skipping the redundant topmost level of BMSSP-26, attacks the right term, but our parameter sweep indicates gains of tens of percent and not the factors.

Relation to earlier measurements. Our conclusion agrees with every study of an implementation that follows the published algorithm: [7] report Dijkstra to be three to four times faster on large instances, [8] find it faster on Lightning Network graphs, and the solver of [17] that follows the paper is reported as two to four times slower. Since our BMSSP-25 has the same structure as the worst-case implementation of [7], their published numbers serve as a check that our implementation is not slow. On random graphs with $m = 3n$ they report 10.7 ms for Dijkstra and 38.2 ms for BMSSP at $n = 2^{16}$, 421 and 1286 ms at 2^{20}, and 2023 and 7703 ms at 2^{22} on an Intel i5-10400F; we measure 6 and 11 ms, 271 and 454 ms, and 1866 and 2805 ms on an Apple M1. Both of our programs are faster in absolute terms, the BMSSP implementation by the larger margin, so our ratios (1.9, 1.7 and 1.5) are lower than theirs (3.6, 3.1 and 3.8). The two series also differ in shape: their ratio falls from 25 at $n = 2^7$ to about three at 2^{14} and then stays between 2.8 and 3.8 up to 2^{25}, whereas ours keeps declining slowly up to 2^{22}. The machines and graph generators differ and we did not run their code, so we cannot say whether this reflects the implementations or the hardware, and the comparison is indicative only. The reports of a practical advantage over Dijkstra that we are aware of [17], [18] concern simplified variants on unit-weight graphs (Section II), and Section VI-H shows no such advantage for the published algorithms on the same graphs.

Threats to validity. All timings come from one machine and one compiler. The implementations follow the pseudocode and use sensible engineering (flat arrays, direct-address tables, no per-call allocation), but they are not tuned to the limit, and the base case of BMSSP-26 uses a node-based search tree because its pseudocode asks for one. The operation counts are independent of these choices only in part: they depend on the heap and selection routines used, and a heap with fewer comparisons per operation would shift Dijkstra's curves downward. Weights are integers; rounding effects of floating-point weights were not studied. The largest instances have 2^{22} vertices, which is far from the regime in which the asymptotic bounds separate, and we make no claim about where a crossover in running time would occur. Finally, the two observations of Section VI-A concern our reading of the pseudocode of [6] and have not been confirmed by its authors.

VIII. Conclusion

We compared Dijkstra's algorithm with the two algorithms of Duan et al. that break the sorting barrier for directed graphs, using independent implementations, eleven real graphs, four synthetic families and counts in the comparison-addition model. Dijkstra's algorithm with a binary heap was the fastest of the three on every instance, by factors between 1.2 and 8.7. In the cost model the picture is more differentiated. Where Dijkstra's frontier is narrow, as on road networks and grids, the new algorithms perform more comparisons and additions at every size. Where it is wide and degrees are bounded, BMSSP-25 performs fewer comparisons than lazy-heap Dijkstra from a few tens of thousands of vertices on (from 10^5 to 10^6 against the DecreaseKey variant), and BMSSP-26 from about a million: the asymptotic advantage is real and observable on synthetic graphs. On real web and social graphs it shows only against the lazy heap and only on the two largest instances. It is not yet an advantage in time, because every arc is scanned several times and each operation is costlier. BMSSP-26 achieves the cheaper pivot finding its analysis promises, but with the prescribed parameters this did not make it faster than BMSSP-25 on most inputs. The validation exposed two places where the published pseudocode of BMSSP-26 departs from its stated invariants; both are easy to repair and neither changed a computed distance. Useful next steps are parameter rules derived from measured constants instead of asymptotic balance, an implementation of BMSSP-26 that avoids the redundant top level and the node-based base case, and measurements on machines large enough to follow the comparison counts beyond 2^{22} vertices.

References

[1] E. W. Dijkstra, "A note on two problems in connexion with graphs," *Numer. Math.*, vol. 1, pp. 269–271, 1959, doi: 10.1007/BF01386390.

[2] M. L. Fredman and R. E. Tarjan, "Fibonacci heaps and their uses in improved network optimization algorithms," *J. ACM*, vol. 34, no. 3, pp. 596–615, Jul. 1987, doi: 10.1145/28869.28874.

[3] J. R. Driscoll, H. N. Gabow, R. Shrairman, and R. E. Tarjan, "Relaxed heaps: An alternative to Fibonacci heaps with applications to parallel computation," *Commun. ACM*, vol. 31, no. 11, pp. 1343–1354, Nov. 1988, doi: 10.1145/50087.50096.

[4] B. Haeupler, R. Hladík, V. Rozhoň, R. E. Tarjan, and J. Tětek, "Universal optimality of Dijkstra via beyond-worst-case heaps," in *Proc. IEEE 65th Annu. Symp. Found. Comput. Sci. (FOCS)*, 2024, pp. 2099–2130, doi: 10.1109/FOCS61266.2024.00125.

[5] R. Duan, J. Mao, X. Mao, X. Shu, and L. Yin, "Breaking the sorting barrier for directed single-source shortest paths," in *Proc. 57th Annu. ACM Symp. Theory Comput. (STOC)*, 2025, pp. 36–44, doi: 10.1145/3717823.3718179.

[6] R. Duan, X. Mao, X. Shu, and L. Yin, "A faster directed single-source shortest path algorithm," in *Proc. 53rd Int. Colloq. Automata, Lang., Program. (ICALP)*, LIPIcs, vol. 374, 2026, pp. 81:1–81:23, doi: 10.4230/LIPIcs.ICALP.2026.81.

[7] L. Castro, T. Clementino, and R. de Freitas, "Implementation and experimental analysis of the Duan et al. (2025) algorithm for single-source shortest paths," 2026, arXiv:2511.03007v3.

[8] D. Valko, R. Paranjpe, and J. Marx Gómez, "An empirical evaluation of BMSSP and Dijkstra's algorithm for the Lightning Network," 2026, arXiv:2509.13448v2.

[9] C. Makowski, W. Guter, T. Russell, and A. Saragih, "BMSSPy: A Python package and empirical comparison of bounded multi-source shortest path algorithm," MIT Center Transp. Logistics, Cambridge, MA, USA, Res. Paper 2025/034, Nov. 2025. [Online]. Available: https://ssrn.com/abstract=5777186

[10] B. V. Cherkassky, A. V. Goldberg, and T. Radzik, "Shortest paths algorithms: Theory and experimental evaluation," *Math. Program.*, vol. 73, no. 2, pp. 129–174, May 1996, doi: 10.1007/BF02592101.

[11] M. Thorup, "Undirected single-source shortest paths with positive integer weights in linear time," *J. ACM*, vol. 46, no. 3, pp. 362–394, May 1999, doi: 10.1145/316542.316548.

[12] S. Pettie and V. Ramachandran, "A shortest path algorithm for real-weighted undirected graphs," *SIAM J. Comput.*, vol. 34, no. 6, pp. 1398–1431, 2005, doi: 10.1137/S0097539702419650.

[13] R. Duan, J. Mao, X. Shu, and L. Yin, "A randomized algorithm for single-source shortest path on undirected real-weighted graphs," in *Proc. IEEE 64th Annu. Symp. Found. Comput. Sci. (FOCS)*, 2023, pp. 484–492, doi: 10.1109/FOCS57990.2023.00035.

[14] S. Yan, "Lossless derandomization for undirected single-source shortest paths and approximate distance oracles," in *Proc. SIAM Symp. Simplicity Algorithms (SOSA)*, 2026, pp. 404–412, doi: 10.1137/1.9781611978964.32.

[15] A. Kadria and L. Roditty, "A faster undirected single-source shortest path algorithm," 2026, arXiv:2609.15247.

[16] L. Castro, T. Clementino, and R. de Freitas. *bmssp: Header-only C++ implementation of the shortest-path BMSSP algorithm*. GitHub. Accessed: Oct. 12, 2026. [Online]. Available: https://github.com/lcs147/bmssp

[17] M. Ryu. *bmssp-python: Python implementation of the BMSSP shortest path algorithm*. (2025). GitHub. Accessed: Oct. 12, 2026. [Online]. Available: https://github.com/bzantium/bmssp-python

[18] J. Webb. *DunMaoSSSP: A Rust implementation of the Duan–Mao et al. algorithm for single-source shortest paths*. (2025). GitHub. Accessed: Oct. 12, 2026. [Online]. Available: https://github.com/alphastrata/DunMaoSSSP

[19] G. N. Frederickson, "Data structures for on-line updating of minimum spanning trees," in *Proc. 15th Annu. ACM Symp. Theory Comput. (STOC)*, 1983, pp. 252–257, doi: 10.1145/800061.808754.

[20] J. W. J. Williams, "Algorithm 232: Heapsort," *Commun. ACM*, vol. 7, no. 6, pp. 347–348, Jun. 1964.

[21] L. J. Guibas and R. Sedgewick, "A dichromatic framework for balanced trees," in *Proc. 19th Annu. Symp. Found. Comput. Sci. (SFCS)*, 1978, pp. 8–21, doi: 10.1109/SFCS.1978.3.

[22] M. Blum, R. W. Floyd, V. Pratt, R. L. Rivest, and R. E. Tarjan, "Time bounds for selection," *J. Comput. Syst. Sci.*, vol. 7, no. 4, pp. 448–461, Aug. 1973, doi: 10.1016/S0022-0000(73)80033-9.

[23] D. R. Musser, "Introspective sorting and selection algorithms," *Softw. Pract. Exper.*, vol. 27, no. 8, pp. 983–993, Aug. 1997.

[24] Anmol, T. Arora, Adhyyan, and P. V. S. Gogna. *bmssp-vs-dijkstra: Implementations, experiments and data for a comparison of Dijkstra's algorithm and the SSSP algorithms of Duan et al.* (2026). GitHub. Accessed: Oct. 12, 2026. [Online]. Available: https://github.com/anmoltutejagithub/bmssp-vs-dijkstra

[25] F. Li, D. Cheng, M. Hadjieleftheriou, G. Kollios, and S.-H. Teng, "On trip planning queries in spatial databases," in *Advances in Spatial and Temporal Databases (SSTD)*, Lecture Notes Comput. Sci., 2005, pp. 273–290, doi: 10.1007/11535331_16.

[26] M. Ansari, 2020, "Real dataset shortest path sample (road networks of the SSTD 2005 collection)," GitHub. [Online]. Available: https://github.com/mehdi-ansari/Real-dataset-shortest-path-sample

[27] T. Brinkhoff, "A framework for generating network-based moving objects," *GeoInformatica*, vol. 6, no. 2, pp. 153–180, Jun. 2002, doi: 10.1023/A:1015231126594.

[28] J. Leskovec and A. Krevl, Jun. 2014, "SNAP datasets: Stanford large network dataset collection," Stanford Univ. [Online]. Available: http://snap.stanford.edu/data

[29] A.-L. Barabási and R. Albert, "Emergence of scaling in random networks," *Science*, vol. 286, no. 5439, pp. 509–512, Oct. 1999, doi: 10.1126/science.286.5439.509.

[30] R. Bellman, "On a routing problem," *Quart. Appl. Math.*, vol. 16, no. 1, pp. 87–90, 1958.